인공지능기초 · 7주차 실습 · 25분

손으로 확인하는 깊은 신경망 학습

강의에서 본 그래프를 내 코드로 다시 그려 보고,
학습 로그만 보고 무엇이 잘못됐는지 진단한다

① 기울기 소실 ② 최적화 ③ 규제화 ④ 디버깅 튜터

오늘의 실습 지도

강의의 세 PART를 하나씩 직접 돌려 본다

0 · 2분
준비
손글씨 숫자 데이터, 실험 도구 두 개

1 · 7분
기울기 소실
10층 신경망의 층별 기울기를 잰다

2 · 6분
최적화
좁은 골짜기에서 네 방법이 경주한다

3 · 7분
규제화
일부러 과적합을 만들고 막아 본다

4 · 3분
디버깅 튜터
수상한 로그 4개를 진단한다

각 실습은 같은 순서로 진행한다: **예측** → **실행** → **결과 읽기** → **값 하나 바꿔 보기**

실습 0 · 준비 (2분)

작은 데이터, 두 개의 도구

데이터: 손글씨 숫자

8×8 이미지 1,797장, 0~9 열 개 클래스.
훈련 1,257장 · 검증 540장. CPU로도 몇 초.

train()이 에폭마다 남기는 기록

훈련 손실 · 검증 손실 · 검증 정확도 ·
1층 기울기 크기 — 실습 4의 진단 단서

make_mlp() — 실험마다 이것만 바꾼다

```
make_mlp(
  depth=10,       # 은닉층 수
  width=64,       # 층의 폭
  act="sigmoid", # 또는 "relu"
  init="small",  # "he", "xavier"
  p_drop=0.0,    # 드롭아웃 확률
  l2=0.0)         # L2 규제 λ
```

코드 읽기 · 실습 0 ① 데이터 준비

숫자 이미지를 신경망이 먹을 수 있게

```
keras.utils.set_random_seed(0)   ①
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")   ②

Xtr, Xva, ytr, yva = train_test_split(
    X, y, test_size=0.3, random_state=0,
    stratify=y)   ③

loss_fn = keras.losses.SparseCategoricalCrossentropy(
    from_logits=True)   ④
```

① 난수 시드 고정. 누가 돌려도 같은 결과가 나와 슬라이드 숫자와 비교할 수 있다.

② 픽셀 값 0~16을 0~1로. 입력 크기가 고르게 작아야 He·Xavier의 "분산 유지" 계산이 맞는다.

③ 훈련 70% · 검증 30%. stratify는 양쪽의 0~9 비율을 똑같이 맞춘다.

④ 라벨이 정수라서 Sparse, 출력층에 softmax가 없어서 from_logits=True. 아무것도 모르면 손실 = ln 10 ≈ 2.30.

코드 읽기 · 실습 0 ② make_mlp()

인자 하나가 강의의 도구 하나

```
INITS = {   # 요약해서 표시
    "small":  RandomUniform(±1/√n_in),   ①
    "xavier": GlorotNormal(),   # 2/(n_in+n_out)
    "he":     HeNormal()}        # 2/n_in
def make_mlp(depth=10, width=64, act="sigmoid",
             init="small", p_drop=0.0, l2=0.0):
    model = keras.Sequential([keras.Input(shape=(64,))]) ②
    for _ in range(depth):
        model.add(layers.Dense(width, activation=act,   ③
          kernel_initializer=INITS[init](n_in),
          kernel_regularizer=L2(l2) if l2 > 0 else None))④
        if p_drop > 0:
            model.add(layers.Dropout(p_drop))   ⑤
    model.add(layers.Dense(10))   ⑥
    return model
```

① 초기화 세 가지. Xavier·He는 강의 PART 1 공식 그대로, small은 소실을 잘 보여 주는 비교군.

② 입력 64칸(8×8 픽셀)에서 시작해 층을 차례로 쌓는 상자.

③ Dense = W·x + b 다음 활성화. depth번 반복 = 은닉층 수.

④ L2: 이 층 가중치의 λ·Σw²를 손실에 더한다 (PART 3).

⑤ Dropout: 훈련 때만 p 비율의 출력을 0으로. 예측 때는 모두 켠다.

⑥ 출력층은 활성화 없이 점수(logit) 10개. softmax는 손실 함수가 맡는다.

코드 읽기 · 실습 0 ③ EpochLogger

에폭이 끝날 때마다 네 가지를 잰다

```
class EpochLogger(keras.callbacks.Callback):  ①
  def on_epoch_end(self, epoch, logs=None):
    first = self.model.layers[0]
    with tf.GradientTape() as tape:  ②
      pred = self.model(self.Xt, training=False) ③
      tl = loss_fn(self.yt, pred)
    g = tape.gradient(tl, first.kernel)  ④
    out = self.model(Xva, training=False)
    vl = loss_fn(yva, out)
    acc = np.mean(np.argmax(out, axis=1) == yva) ⑤
    for k, v in zip(self.hist,
                    (tl, vl, acc, tf.norm(g))):  ⑥
      self.hist[k].append(float(v))
```

① 콜백: fit()이 정해진 순간(여기서는 에폭 끝)마다 불러 주는 함수.

② GradientTape: 안의 계산을 녹화했다가 거꾸로 미분한다. 6주차 역전파가 이 한 줄.

③ training=False: 드롭아웃을 끄고 L2 벌점 없이 잰다. 그래야 설정끼리 공정하게 비교된다.

④ 첫 층 가중치에 대한 ∂L/∂W. 실습 4 로그의 grad(1층) 칸.

⑤ argmax = 점수가 가장 큰 클래스 = 예측. 맞힌 비율이 정확도.

⑥ 훈련 손실 · 검증 손실 · 정확도 · 기울기 크기를 hist에 쌓는다.

코드 읽기 · 실습 0 ④ train()

compile로 정하고, fit으로 돌린다

```
def train(model, opt, epochs=30, batch=64,
          Xt=Xtr, yt=ytr, patience=None):
    model.compile(optimizer=opt, loss=loss_fn)   ①
    log = EpochLogger(Xt, yt)
    cbs = [log]
    if patience is not None:   ②
        cbs.append(keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=patience,
            restore_best_weights=True))   ③
    model.fit(Xt, yt, validation_data=(Xva, yva),   ④
              epochs=epochs, batch_size=batch,
              shuffle=True, verbose=0, callbacks=cbs)
    return log.hist
```

① compile: 옵티마이저(PART 2)와 손실을 정한다. 같은 모델에 옵티마이저만 바꿔 끼우면 된다.

② patience를 줄 때만 조기 종료를 켠다.

③ 검증 손실이 patience번 연속 나아지지 않으면 멈추고, 가장 좋았던 에폭으로 되돌린다. 강의의 patience 표 그대로.

④ fit: 자료를 섞어 batch개씩 잘라 학습하고, 에폭마다 검증 자료로 잰다.

fit() 안에서 미니배치마다

예측 → 손실 → 기울기 → 가중치 수정 (6주차 4단계). 1,257장 ÷ 64 = 20번 × 30에폭 = 600번 수정.

실습 1 · 기울기 소실 (7분)

학습 전 첫 역전파, 층별 기울기를 잰다

비교할 두 신경망 (은닉층 10개)

```
A = make_mlp(act="sigmoid",
             init="small")
B = make_mlp(act="relu",
             init="he")

with tf.GradientTape() as tape:
  loss = loss_fn(y, model(X))
tape.gradient(loss,
  [l.kernel for l in Dense 층])
```

🎾 먼저 예측하기

A에서 1층(입력 쪽) 기울기는 10층(출력 쪽)의 몇 분의 1일까?

① 4분의 1

② 1,000분의 1

③ 100만분의 1 (강의 계산: 0.25^{10})

④ 그보다 훨씬 작다

코드 읽기 · 실습 1 layer_grad_norms()

층마다 기울기의 크기를 숫자 하나로

```
def layer_grad_norms(model):
    dense = [l for l in model.layers
             if isinstance(l, layers.Dense)][:-1]   ①
    with tf.GradientTape() as tape:
        loss = loss_fn(ytr, model(Xtr, training=False))②
    grads = tape.gradient(loss,
              [l.kernel for l in dense])   ③
    return [float(tf.norm(g)) for g in grads]   ④
keras.utils.set_random_seed(0)   ⑤
g_sig = layer_grad_norms(
    make_mlp(act="sigmoid", init="small"))
keras.utils.set_random_seed(0)
g_relu = layer_grad_norms(
    make_mlp(act="relu", init="he"))
plt.yscale("log")   ⑥
```

① 출력층을 뺀 은닉층 10개의 Dense만 고른다. [:-1]이 마지막 하나를 뺀다.

② fit 없이 순전파 한 번. 학습 전 처음 상태의 기울기를 본다.

③ kernel = 가중치 행렬 W (편향 b 제외). 10개 층의 기울기를 한 번에 구한다.

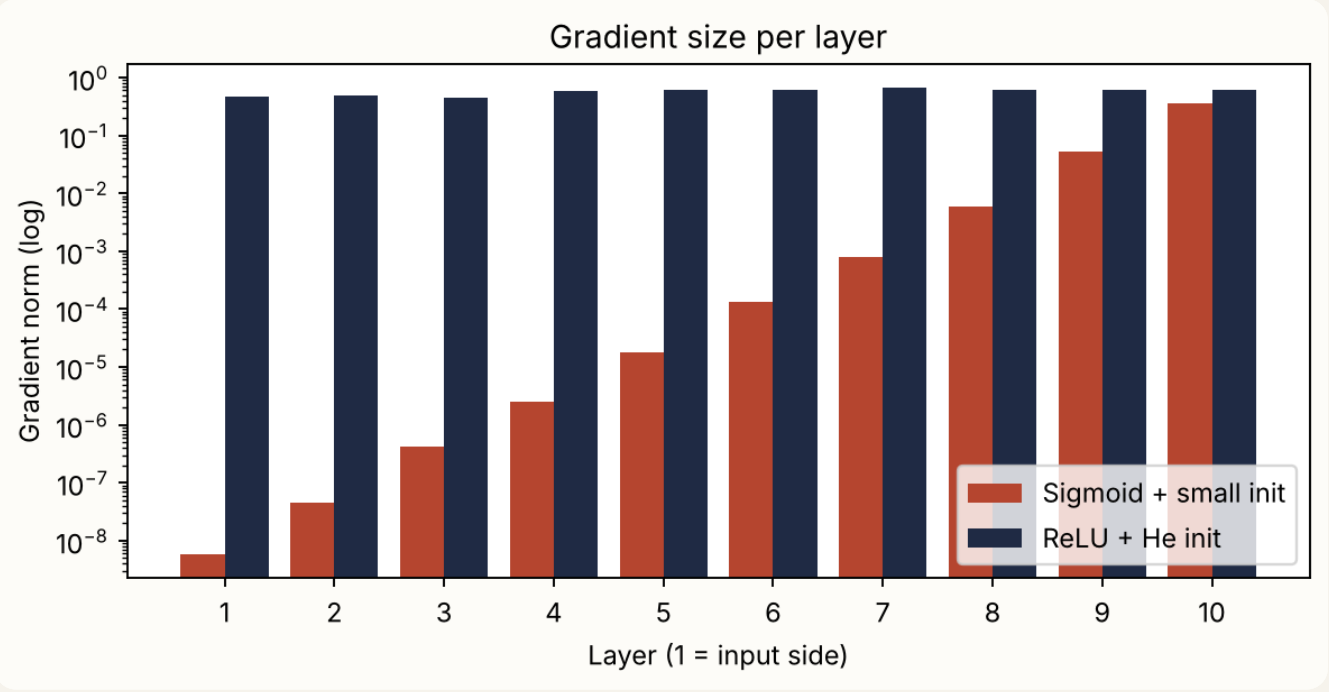
④ 노름 $\sqrt{\Sigma g^2}$: 기울기 행렬의 크기를 숫자 하나로 줄인다.

⑤ 같은 시드 → 같은 출발 조건. 다른 것은 활성화와 초기화 뿐이다.

⑥ 값이 10^{-8}부터 10^{-1}까지 퍼져 있어 로그 축이 아니면 작은 막대가 보이지 않는다.

실습 1 · 결과

시그모이드의 1층은 신호를 거의 받지 못한다



1층 ÷ 10층 기울기

시그모이드 1.6×10^{-8}

ReLU+He 0.78

SGD 30에폭 후 검증 정확도

시그모이드 10.2%

ReLU+He 95.9%

손실 2.30 = ln 10 → 열 개를 똑같이 찍고 있다

바꿔 보기 init="xavier"로 바꾸면? · depth=3으로 줄이면?

실습 2 · 최적화 (6분)

좁고 긴 골짜기에서 40걸음 경주

경주로

$$L = 0.05 \cdot w_1^2 + 2 \cdot w_2^2$$

가로로 길고 세로로 가파르다 (경사 40배 차이).
출발 (−9, 1.5) → 결승 (0, 0)

선수	설정
SGD	lr = 0.45
Momentum	lr = 0.08, β = 0.9
RMSprop	lr = 0.3, ρ = 0.9
Adam	lr = 0.3

🔑 먼저 예측하기

40걸음 뒤 결승점에 가장 가까운 선수는?

강의 장면을 떠올려 보자:

- SGD는 가파른 벽 사이를 왔다 갔다
- 모멘텀은 같은 방향이면 가속
- RMSprop은 변수마다 보폭을 따로

코드 읽기 · 실습 2 run_path()

fit() 속 한 걸음을 꺼내서 본다

```
def valley(w):
    return 0.05 * w[0] ** 2 + 2.0 * w[1] ** 2   ①
w = keras.Variable(np.array([-9.0, 1.5],
                            dtype="float32"))   ②
opt = keras.optimizers.SGD(learning_rate=0.08,
                           momentum=0.9)   ③
for _ in range(40):
    with tf.GradientTape() as tape:
        loss = valley(w)
    g = tape.gradient(loss, w.value)   ④
    opt.apply_gradients([(g, w)])   ⑤
    path.append(w.numpy().copy())
```

① 기울기 = $(0.1 \cdot w_1, 4 \cdot w_2)$. 세로가 가로보다 40배 가파르다.

② 가중치 2개짜리 "모델". 출발점 (−9, 1.5).

④ 손으로 미분하지 않아도 GradientTape가 g를 구한다.

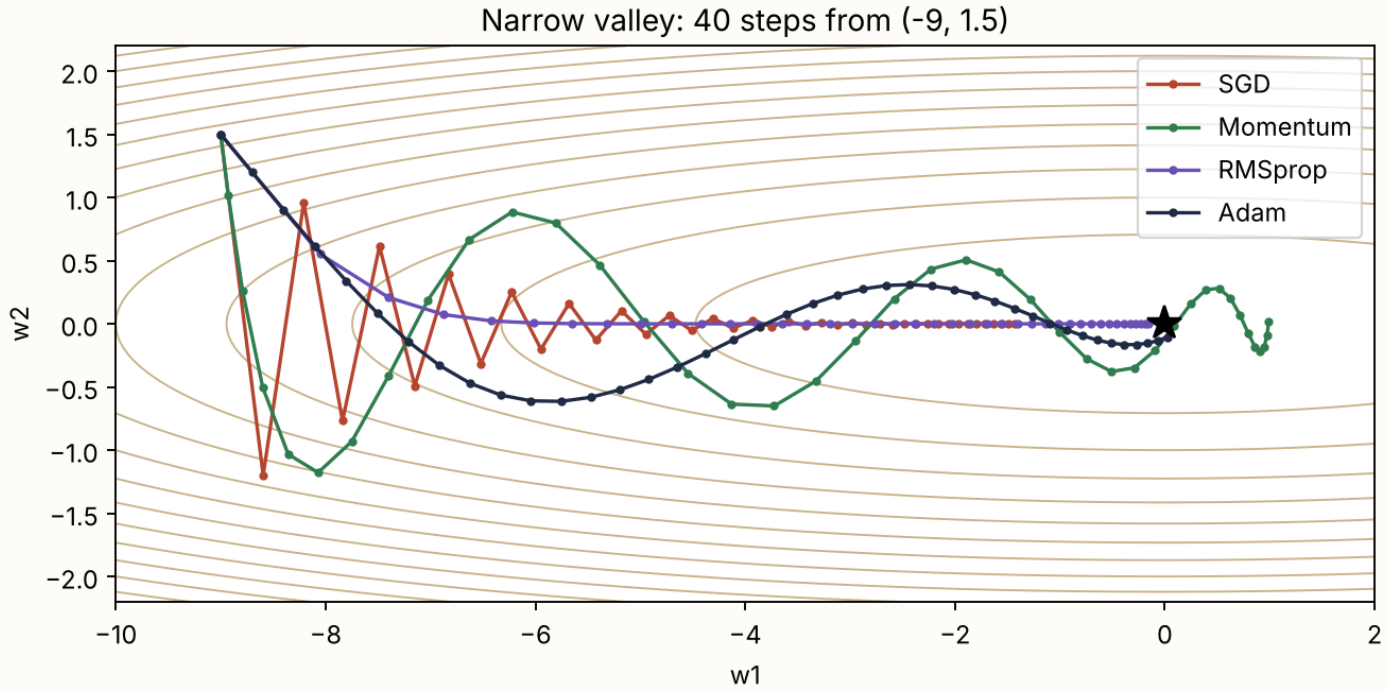
⑤ 옵티마이저 공식대로 한 걸음. fit()은 이것을 미니배치마다 한다.

③ Keras 인자 ↔ 강의 수식

Keras	강의
SGD(learning_rate=η)	$w \leftarrow w - \eta \cdot g$
SGD(momentum=0.9)	$v \leftarrow \beta \cdot v + g$
RMSprop(rho=0.9)	$s \leftarrow \rho \cdot s + (1-\rho) \cdot g^2$
Adam()	$\beta_1=0.9$, $\beta_2=0.999$

실습 2 · 결과

방향을 기억하거나, 보폭을 나누거나



40걸음 뒤 손실

선수	손실
SGD	0.102
Momentum	0.050
Adam	0.022
RMSprop	0.001

바꿔 보기 SGD lr 0.45 → 0.55 · momentum 0.9 → 0.0 · 모멘텀은 왜 결승점을 지나칠까?

실습 3 · 과적합과 규제화 (7분)

일부러 "외우기 쉬운" 학생을 만든다

외우기 쉬운 조건

- 훈련 자료 **200장**뿐
- 그중 **20%는 정답 라벨이 틀림** (라벨링 실수)
- 큰 신경망: 은닉층 2개 × 512, 150에폭

설정	코드에서 바뀌는 것
규제 없음	—
L2	l2=0.005
드롭아웃	p_drop=0.5
조기 종료	patience=10

🔑 먼저 예측하기

규제가 없으면 틀린 라벨 32장은 어떻게 될까?

훈련 손실이 0에 가까워진다면,
신경망은 틀린 답까지 "정답"으로 외운 것이다.
그때 검증 손실은?

코드 읽기 · 실습 3 라벨 흔들기와 통제 실험

훈련 자료만 흔들고, 한 번에 하나만 바꾼다

```
n_small, noise = 200, 0.2; rng = np.random.default_rng(1)
Xs, ys = Xtr[:n_small], ytr[:n_small].copy()   ①
flip = rng.random(n_small) < noise   ②
ys[flip] = rng.integers(0, 10, flip.sum())   ③
configs = {   ④
  "None":    dict(p_drop=0.0, l2=0.0,   patience=None),
  "L2":      dict(p_drop=0.0, l2=0.005, patience=None),
  "Dropout": dict(p_drop=0.5, l2=0.0,   patience=None),
  "Early":   dict(p_drop=0.0, l2=0.0,   patience=10)}
for name, c in configs.items():
    keras.utils.set_random_seed(0)
    m = make_mlp(depth=2, width=512, act="relu",
                 init="he", p_drop=c["p_drop"], l2=c["l2"])
    reg[name] = train(m, keras.optimizers.Adam(1e-3), ⑤
        epochs=150, batch=32, Xt=Xs, yt=ys,
        patience=c["patience"])
```

① 훈련 자료 앞 200장만 쓴다. copy()로 원본 ytr은 건드리지 않는다.

② 자료마다 0~1 난수를 뽑아 0.2보다 작으면 흔들 대상. 약 20%, 이번엔 32장.

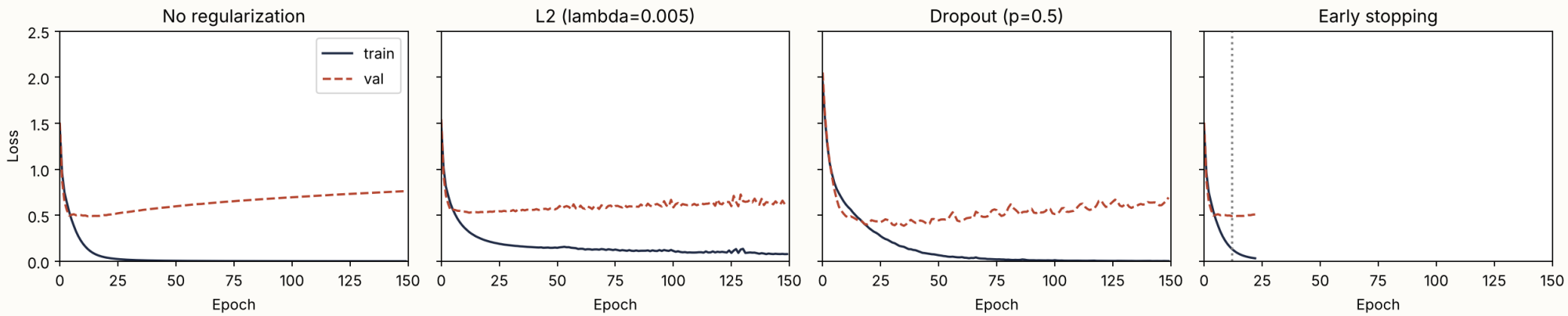
③ 그 자리에 0~9 무작위 라벨을 넣는다. 검증 자료는 깨끗한 원본 그대로.

④ 네 설정은 규제 한 가지씩만 다르다. 시드·구조·옵티마이저·에폭은 모두 같다.

⑤ 학습은 흔든 Xs, ys로, 검증은 train() 안에서 깨끗한 Xva, yva로. 그래서 "외운 것"이 검증 손실에 드러난다.

실습 3 · 결과

세 가지 규제 모두 검증 손실을 끌어내렸다



설정	훈련 손실	검증 손실	검증 정확도
규제 없음	0.000	0.762	83.5%
L2 (λ=0.005)	0.078	0.623	81.7%
드롭아웃 (0.5)	0.002	0.693	82.4%
조기 종료 (13에폭 복원)	0.134	0.489	85.0%

실습 4 · 디버깅 튜터 (3분 + 과제)

정체를 숨긴 학습 로그 A~D, 무엇이 잘못됐나?

STEP 1 · 묻기

로그 하나를 복사해 생성형 AI에게 보여준다

STEP 2 · 검증하기

AI의 진단을 강의 마지막 장의 증상–원인–처방 표와 대조한다

STEP 3 · 고치기 (과제)

처방을 코드 한 줄로 적용해 다시 돌리고 로그를 비교한다

AI에게 이렇게 묻는다

"다음은 신경망 학습 로그다. 의심되는 문제와 처방은? **근거가 된 숫자**를 로그에서 짚어서 설명해 줘."

AI 답을 믿기 전에 확인한다

AI가 짚은 숫자가 로그에 정말 있는가? 표의 증상과 맞는가? 처방이 실제로 고쳤는가?

코드 읽기 · 실습 4 print_log()

로그의 다섯 칸을 읽는 법

```
def print_log(code, h, every=None):
    every = every or max(1, len(h["train"]) // 10)
    for i in range(0, len(h["train"]), every):
        print(i+1, h["train"][i], h["val"][i],
              h["acc"][i], h["g_first"][i])  # 요약
```

기준선 두 개를 먼저 외운다

손실 **2.30** = ln 10 = 열 개 중 아무거나 찍기
정확도 **0.10** = 열 개 중 하나 맞히기

칸	무엇을 잰 값인가	이렇게 읽는다
train_loss	학습에 쓴 자료의 손실	0에 가까울수록 훈련 자료를 많이 외웠다
val_loss	처음 보는 검증 자료의 손실	일반화 성적표. train_loss와 벌어지는지 본다
val_acc	검증 자료의 정답 비율	0.10 근처면 찍기 수준
grad(1층)	첫 층 가중치 기울기의 크기	아주 작으면 앞쪽 층이 못 배우는 중, nan이면 숫자가 터졌다

실습 4 · 정답 공개 (진단 후에)

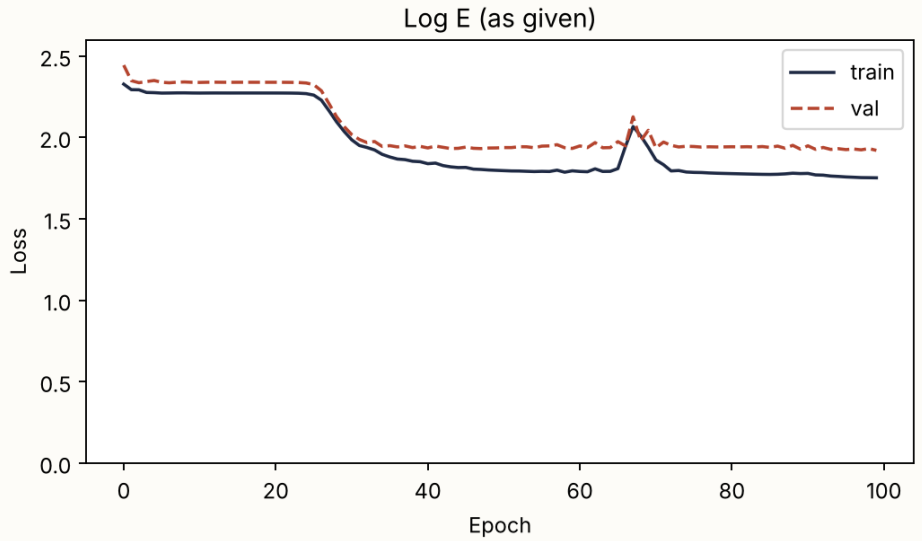
근거는 언제나 로그 속 숫자에 있다

로그	로그 속 근거	원인	처방 (한 줄 수정)
A	훈련 손실 1.46 → 0.004, 검증 손실은 13에폭 0.49가 최저, 이후 0.61까지 상승	과적합	p_drop=0.5 또는 patience=10
B	손실 2.30에서 20에폭 내내 정체, 1층 기울기 ≈ 3×10^{-9}	기울기 소실	act="relu", init="he"
C	1에폭 손실 2.45, 3에폭부터 nan	기울기 폭발 (학습률 과대)	lr 1.5 → 0.05
D	손실이 2.19 → 0.42로 매 에폭 조금씩만 감소, 20에폭에도 정확도 상승 중	보폭 문제 (너무 느림)	모멘텀 0.9 추가 또는 Adam

AI가 "학습률을 낮추라"고만 했다면? — 그 처방이 **어느 로그에는 맞고 어느 로그에는 오히려 해로운지** 따져 보자.

7주차 과제 · 1/3

망가진 학습 고치기: 로그 E



노트북 맨 아래 과제 셀

```
def homework(act="sigmoid", init="small",
             p_drop=0.0, l2=0.0, patience=None):
    keras.utils.set_random_seed(0)
    m = make_mlp(depth=6, width=256, act=act,
                 init=init, p_drop=p_drop, l2=l2)
    return train(m, keras.optimizers.Adam(1e-3),
                 epochs=100, batch=32, Xt=Xs, yt=ys,
                 patience=patience)
hE = homework()   # 로그 E
```

규칙 함수 안은 고치지 않는다. 호출할 때 **인자만** 바꾼다.

데이터 실습 3의 라벨을 흔든 200장 (Xs, ys)

7주차 과제 · 2/3

진단하고, 한 줄 고치고, 다시 진단한다

단계 1 · 진단

로그 E의 문제를 진단한다. 근거가 된 숫자를 로그에서 **2개 이상** 짚는다.

바꿀 인자: 없음

단계 2 · 1차 처방

처방을 적용해 로그 E1을 만든다. 고쳐진 뒤 **새로 나타난 증상**을 진단한다.

`act, init`

단계 3 · 2차 처방

규제 하나를 골라 **강도를 2가지 이상** 시험해 로그 E2를 만들고, E1과 표로 비교한다.

`p_drop, l2, patience` 중 하나

단계 4 · AI 검증

로그 E를 생성형 AI에 보여 주고, AI의 진단과 내 진단이 같은지 다른지 1~2문장으로 적는다.

힌트

실습 3(은닉층 2개)에서 잘 듣던 강도가 은닉층 6개짜리에서도 최선일까? 예: p_drop 0.2 / 0.3 / 0.5, l2 0.001 / 0.005, patience 5 / 10

7주차 과제 · 3/3

제출물과 채점 기준

항목	제출할 것	배점
단계 1 진단	원인 한 가지 + 로그 E에서 짚은 근거 숫자 2개 이상	20
단계 2 1차 처방	바꾼 인자, 로그 E1, 새로 나타난 증상과 그 근거	30
단계 3 2차 처방	강도 2가지 이상의 검증 손실·정확도 비교표, 고른 값과 이유	30
단계 4 AI 검증	AI 답변 캡처 + 내 진단과 같은 점·다른 점 1~2문장	10
그래프	E, E1, E2의 plot_hist 그래프 3장	10

제출 형식

실행 결과가 남아 있는 .ipynb 1개. 답은 마크다운 셀에 쓴다. 마감: [월 / 일]

성공 기준

E2의 검증 손실이 E1의 마지막 검증 손실보다 낮으면 처방 성공.

7주차 실습 정리

오늘 내 코드로 확인한 세 가지

기울기 소실

$1.6\times10^{-8} \rightarrow 0.78$

시그모이드 대신 ReLU+He로 1층 기울기가 살아났다

최적화

0.102 → 0.001

같은 40걸음, 보폭을 변수마다 나누자 손실이 100분의 1로

규제화

0.762 → 0.489

틀린 라벨을 외우기 전에 멈추자 검증 손실이 내려갔다

과제

망가진 학습 고치기 (로그 E) — 진단 → act·init 처방 → 규제 강도 비교 → AI 진단과 대조. 노트북 맨 아래 과제 셀에서 시작한다.

다음 주 · **CNN** — 오늘 쓴 ReLU · Adam · 드롭아웃이 그대로 들어간다